CPE 334 | Individual Sprint 1

TokTickIT

Lab 1 full-stack vertical slice

Stack	React, TypeScript, Vite, Bootstrap, Express, Prisma, PostgreSQL
Workflow	Bun workspaces, feature branches, GitHub Issues, Pull Requests
Evidence	Tests, API contracts, database seed, and human-checked App Demo

Built from one LaTeX source with XeLaTeX.

Contents

1 Answer Part 1: Git Use with Engineering Workflow

The repository uses four parent Lab 1 issues and native sub-issues. Work is performed on feature branches and targets `lab1-staging`; the stable branch is `main`.

Parent issue	Feature branch	Pull Request
#13 foundation	`feature/1-project-foundation`	PR #44
#14 health	`feature/2-health-check`	PR #46
#15 categories	`feature/3-category-seed`	PR #45
#16 category list	`feature/4-category-list`	PR #47

Student and peer reviewer

Student
GUNTEE DOUNGMANEE
ID: 67070501003
GitHub: ovenmakeme-heat

Peer reviewer
POLWARIT WATTHANA-HEMMARAT
ID: 67070501067
GitHub: MadMax168

- Repository: `https://github.com/ovenmakemeheat/toktickit`
- Project board: `https://github.com/users/ovenmakemeheat/projects/1/views/2`
- The parent issue checklists are the acceptance source; native sub-issues carry implementation, testing, and evidence slices.
- Positive peer-review records for PRs #44-#47 are audited with the author responses.
- Full review details are in `docs/lab-01/reviewer.md`; PR #47 remains open for human workflow completion.
- Unresolved automated threads on PRs #44, #45, and #47 are recorded separately and are not peer-review rejections.

2 Answer Part 2: Tests

The test matrix is kept in `docs/lab-01/tests.md`. The application boundary is tested through the exported Express app and user-observable React behavior.

ID	Tool	Proof	Status
API-01	Supertest	Health returns HTTP 200 and the exact TokTickIT API payload.	Passing
API-02	Supertest	Categories return seeded ID/name pairs in ascending ID order.	Passing
UI-01	Vitest	TokTickIT heading and Check System action render.	Passing
UI-02	Vitest	Checking transitions to Online and renders returned categories.	Passing
UI-03	Vitest	API failure clears stale categories and shows the safe error.	Passing

Verification command: `bun run verify`. It covers Biome, Lefthook validation, Prisma validation, type checks, both workspace test suites, and both workspace builds.

3 Answer Part 3: AI Use and Reflection

Tool and model

Tool	OpenAI Codex coding agent in ChatGPT
Model	GPT-5
Supporting tools	GitHub connector, browser-use CLI, Bun, XeLaTeX, and Poppler

Selected prompts and effects

- **P1:** *read @docs/requirements, $grill-with-docs.* Loaded the requirements and used the grilling workflow to identify scope and risks.
- **P2:** *now follow the plan, first create the issues.* Converted the requirements into the Lab 1 issue structure.
- **P3:** *all subissues should be inside the main 4 issues.* Kept exactly four parent issues and attached implementation work as native sub-issues.
- **P4:** *please keep insturction in AGENTS.md strictly about don't close any PR or main issue you can only do check the checklists or closing subissue, closing main issue / PR close is human job.* Established the human-only rules for pull-request and main-issue state changes.
- **P5:** *add biome, left-hook, prisma validation on git sub-issues of issue 1.* Added the requested quality gates and connected them to the foundation work.
- **P6:** *yes, and pdf should be built in latex.* Chose a reproducible XeLaTeX report source and PDF build command.
- **P7:** *do capture those for me from http://localhost:5173/.* Defined the required loading, success, and failure evidence for the running app.
- **P8:** *do use browseruse cli.* Used browser-use CLI to verify the visible UI states and save the captures.
- **P9:** *this is student - reviewer details Student ... also checkout all pr and review records; update reviewer part.* Audited PRs #44-#47 and recorded the student, peer, review comments, responses, and unresolved automated threads.

Reflection

The prompts became more useful as they added concrete constraints: Bun, the server-owned database stack, four parent issues, native sub-issues, acceptance criteria, and human-only pull-request state changes. Those constraints kept the implementation aligned with Lab 1 instead of expanding into later features.

The AI-assisted workflow still required human-in-the-loop checks. The browser capture was inspected visually; an initial loading attempt was detected as a completed success state and recaptured correctly. The final PDF was rendered page by page and checked for readable evidence, clipping, overlap, and excessive empty space. GitHub review records were also checked directly so positive peer comments were not mislabeled as formal approvals, and unresolved automated threads were preserved rather than reported as fixed.

4 Answer Part 4: App Demo

The system check makes two relative API requests through the Vite proxy. The success state proves that health is online and that the four categories came from PostgreSQL. The failure state proves that the user receives a safe error and stale categories are cleared.

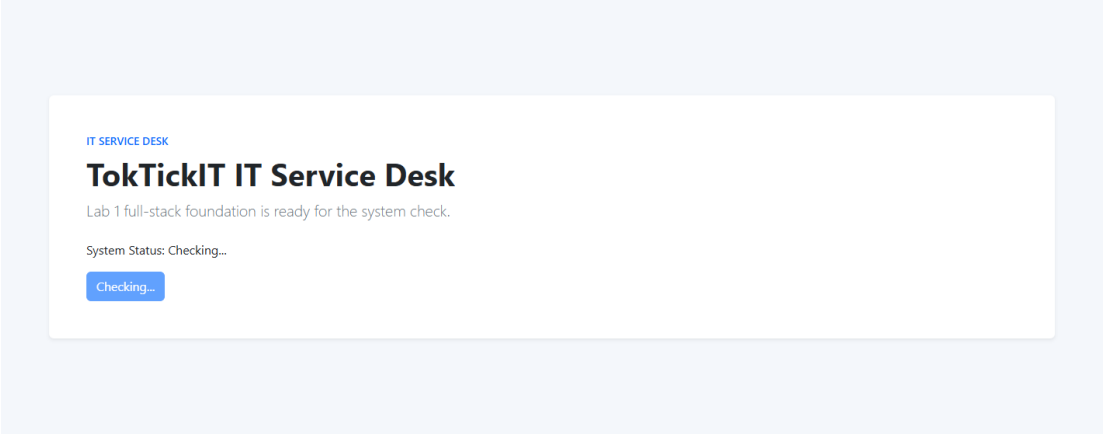

Figure 1: Loading state: the button and status show that the system check is in progress.

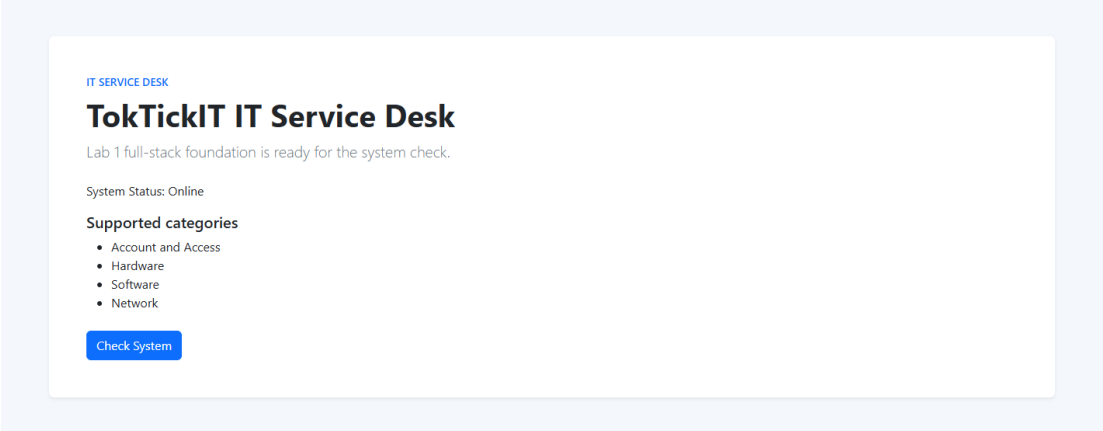

Figure 2: Success state: Online and the database-backed categories are visible in order.

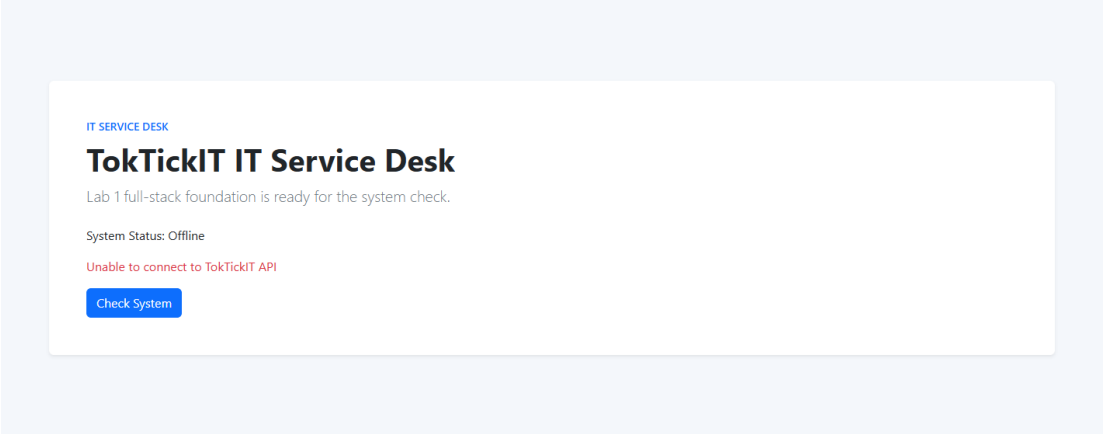

Figure 3: Failure state: the API is unavailable and the user-safe error is shown.

The capture protocol and visual QA checklist are in `docs/lab-01/evidence/capture-checklist.md`. Each final page must be checked for clipped text, unreadable evidence, accidental blank space, and incorrect state labels.